

Task 1:

At first we need to generate the two sample tables: for that the following code was used. And the base line code was generated using genAI and modified as required.



bidhan-pokhrel Update generate_tables.py

Code

Blame

44 lines (37 loc) · 1.52 KB

```
1  import csv
2  import random
3  from datetime import datetime, timedelta
4
5  # Function to generate random dates
6  def random_date(start_date, end_date):
7      time_between = end_date - start_date
8      days = time_between.days
9      random_days = random.randrange(days)
10     return start_date + timedelta(days=random_days)
11
12 # Define date range for DOB (1985-01-01 to 2000-12-31)
13 start_date = datetime(1985, 1, 1)
14 end_date = datetime(2000, 12, 31)
15
16 # Sample names and course IDs
17 names = ["Alice Smith", "Bob Johnson", "Charlie Brown", "Diana Wilson", "Emma Davis",
18         "Frank Miller", "Grace Lee", "Henry Clark", "Isabella Adams", "Jack Taylor"]
19 course_ids = ["CS101", "MATH201", "PHY301", "ENG401", "BIO501"]
20
21 # Generate Table A (StudentId, Name, DOB)
22 table_a = []
23 for i in range(1, 101): # 100 students
24     dob = random_date(start_date, end_date).strftime('%Y-%m-%d')
25     table_a.append([i, random.choice(names), dob])
26
27 # Write Table A to CSV
28 with open('table_a.csv', 'w', newline='') as f:
29     writer = csv.writer(f)
```

```

30     writer.writerow(['StudentId', 'Name', 'DOB'])
31     writer.writerows(table_a)
32
33     # Generate Table B (StudentId, CourseId, Grade)
34     table_b = []
35     for i in range(1, 101): # Each student has 1-3 courses
36         num_courses = random.randint(1, 3)
37         for _ in range(num_courses):
38             table_b.append([i, random.choice(course_ids), round(random.uniform(60.0, 100.0), 1)])
39
40     # Write Table B to CSV
41     with open('table_b.csv', 'w', newline='') as f:
42         writer = csv.writer(f)
43         writer.writerow(['StudentId', 'CourseId', 'Grade'])
44         writer.writerows(table_b)

```

Then, we need to code the mapper and reducer for the data set that we have generated using DeepSeek and we modified the code as required. (Though Gen AI gives the most of the code parts but it never gives the perfect codes so, we have to do some tweaks to make it work for us.)

Following are the codes of mapper.py for task 1:

[CST4070 / mapper.py](#) 



[bidhan-pokhrel](#) Update mapper.py

Code

Blame

39 lines (33 loc) · 1.34 KB

```

1  #!/usr/bin/env python3
2
3  import sys
4  from datetime import datetime
5
6  def mapper():
7      for line in sys.stdin:
8          line = line.strip()
9          parts = line.split(',')
10
11         if len(parts) == 3:
12             # Attempt to parse as Table A (StudentId, Name, DOB)
13             try:
14                 student_id = int(parts[0])
15                 name = parts[1]
16                 dob_str = parts[2]
17                 # Try to parse DOB to confirm it's Table A
18                 datetime.strptime(dob_str, '%Y-%m-%d')
19
20                 # Filter by DOB >= 1995-01-01
21                 if dob_str >= '1995-01-01':
22                     print(f"{student_id}\tA\t{name}\t{dob_str}")
23                     continue # Processed as Table A, move to next line
24             except ValueError:
25                 # If DOB parsing failed, it's likely not Table A format, try Table B
26                 pass # Continue to the next 'try' block
27

```

```

27
28         # Attempt to parse as Table B (StudentId, CourseId, Grade)
29         try:
30             student_id = int(parts[0])
31             course_id = parts[1]
32             grade = float(parts[2]) # Grade should be a float
33             print(f"{student_id}\tB\t{course_id}\t{grade}")
34         except ValueError:
35             # Skip header or malformed lines that don't fit either pattern
36             continue
37
38     if __name__ == "__main__":
39         mapper()

```

And the code for reducer.py for task 1 is:

[CST4070 / reducer.py](#) 



bidhan-pokhrel Update reducer.py

Code

Blame

45 lines (34 loc) · 1.44 KB

```

1  #!/usr/bin/env python3
2
3  import sys
4
5  def reducer():
6      current_student_id = None
7      student_info = {}
8      course_info = []
9
10     # Print headers first
11     # print("StudentId,Name,CourseId,Grade")
12
13     for line in sys.stdin:
14         line = line.strip()
15         parts = line.split('\t')
16
17         student_id = int(parts[0])
18         source_table = parts[1]
19         data = parts[2:]
20
21         if current_student_id and student_id != current_student_id:
22             # Process the previous student's data
23             if 'name' in student_info: # Ensure we have student details from Table A
24                 for course_data in course_info:
25                     print(f"{current_student_id},{student_info['name']},{course_data['course_id']},{course_data['grade']}")
26

```

```

26
27         # Reset for the new student
28         student_info = {}
29         course_info = []
30
31         current_student_id = student_id
32
33         if source_table == 'A':
34             student_info['name'] = data[0]
35             student_info['dob'] = data[1] # Keep DOB for completeness, though filtered in mapper
36         elif source_table == 'B':
37             course_info.append({'course_id': data[0], 'grade': data[1]})
38
39         # Process the last student's data
40         if current_student_id and 'name' in student_info:
41             for course_data in course_info:
42                 print(f"{current_student_id},{student_info['name']},{course_data['course_id']},{course_data['grade']}")
43
44     if __name__ == "__main__":
45         reducer()

```

Gen AI prompt and process:

For generating two tables:

Step 1: I uploaded the sample image:

Studentid	Name	DOB
M575757	Alice	1994-01-05
M212121	Tom	1993-02-07
M989898	John	1995-06-02

Studentid	Courseid	Grade
M575757	CSD1414	pass
M575757	CSD5050	distinction
M575757	CSD5566	merit
M212121	CSD1414	distinction
M212121	CSD5050	distinction
M212121	CSD5566	distinction
M989898	CSD5050	merit
M989898	CSD5566	distinction

Step 2: I gave the prompt as: Write a Python script that generates two CSV files as below. The csv dataset should have 100 rows.

 image.png
PNG 82.73KB

Write a Python script that generates two CSV files as below. The csv dataset should have 100 rows.



For generating mapper and reducer:

Step1: I uploaded the sql script:

```
select StudentId, Name, CourseId, Grade
from A
join B on A.StudentId = B.StudentId
where DOB >= 1995-01-01
```

Step 2: I gave prompt as : Generate mapper and reducer in python code that will run for two datasets that is generated by the above code and gives the exact same result as the sql query as provided. Generate two different files as mapper.py and reducer.py. The mapper.py and reducer.py should be able to run on Hadoop system as well as in local machine or cluster.

image.png
PNG 9.66KB

Generate mapper and reducer in python code that will run for two datasets that is generated by the above code and gives the exact same result as the sql query as provided. Generate two different files as mapper.py and reducer.py. The mapper.py and reducer.py should be able to run on Hadoop system as well as in local machine or cluster.



So, for now the files needed for task one is ready. We have files as: generate_tables.py, mapper.py and reducer.py

Now time to run.

To Run the machine locally we can use our machines that we make on clusters that will have Linux OS, or the same thing can be done using windows command prompt with slight modifications in code like: instead of “cat” we need to use “type” and few more modifications as shown below:

```
C:\WINDOWS\system32\cmd.exe
C:\Users\Dell\Desktop\MDX\CST4070 KD\assignments\Challenges\Final challenge\task 1>python generate_tables.py
C:\Users\Dell\Desktop\MDX\CST4070 KD\assignments\Challenges\Final challenge\task 1>type table_a.csv table_b.csv | python mapper.py | sort | python reducer.py > output.csv
table_a.csv
table_b.csv

C:\Users\Dell\Desktop\MDX\CST4070 KD\assignments\Challenges\Final challenge\task 1>type output.csv
StudentId,Name,CourseId,Grade
10,Frank Miller,CS101,93.6
100,Charlie Brown,BIO501,60.8
100,Charlie Brown,PHY301,85.2
11,Diana Wilson,BIO501,88.5
11,Diana Wilson,CS101,72.8
11,Diana Wilson,ENG401,92.2
```

Image: Testing mapper and reducer in cmd on windows.

C > Desktop > MDX > CST4070 KD > assignments > Challanges > Final challenge > task 1

Name	Date modified	Type	Size
 cmd.bat	7/9/2025 2:04 PM	Windows Batch File	1 KB
 generate_tables.py	7/8/2025 9:22 PM	Python File	2 KB
 mapper.py	7/8/2025 9:22 PM	Python File	2 KB
 output.csv	7/9/2025 3:06 PM	Microsoft Excel Co...	3 KB
 reducer.py	7/9/2025 2:17 PM	Python File	2 KB
 table_a.csv	7/9/2025 3:06 PM	Microsoft Excel Co...	3 KB
 table_b.csv	7/9/2025 3:06 PM	Microsoft Excel Co...	4 KB

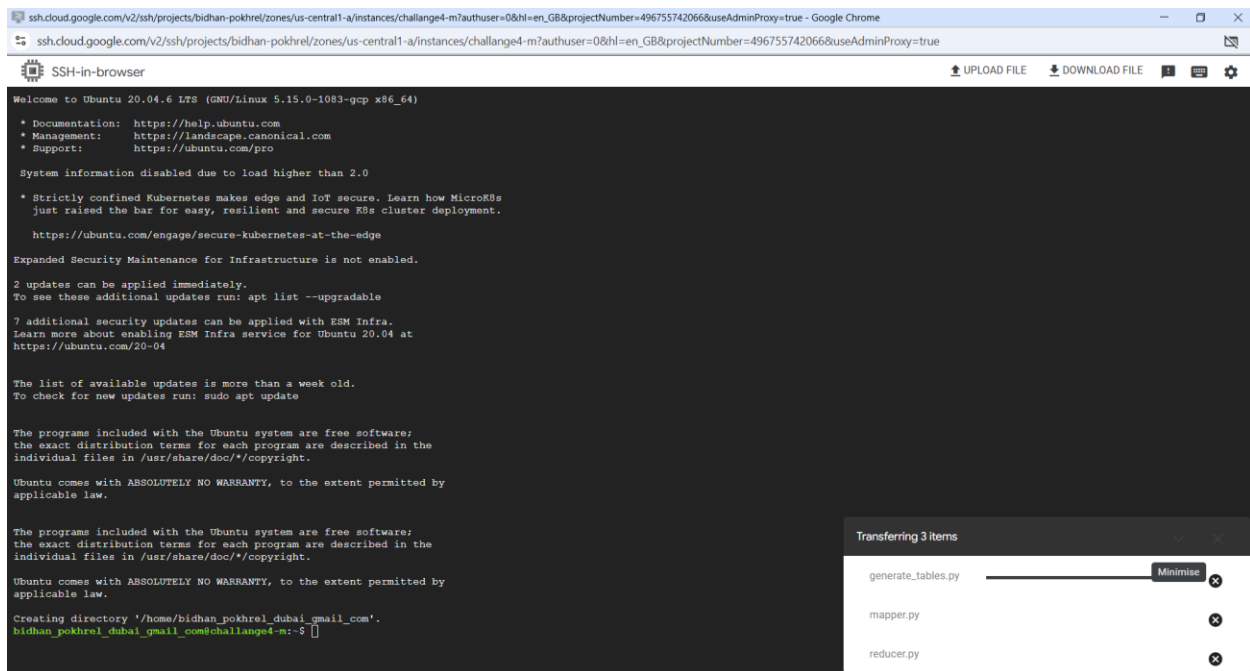
Image: Files that I have on my local windows machine.

```
C:\Users\Dell\Desktop\MDX\CST4070 KD\assignments\Challanges\Final challenge\task 1>type output.csv
StudentId,Name,CourseId,Grade
10,Frank Miller,CS101,93.6
100,Charlie Brown,BIO501,60.8
100,Charlie Brown,PHY301,85.2
11,Diana Wilson,BIO501,88.5
11,Diana Wilson,CS101,72.8
11,Diana Wilson,ENG401,92.2
14,Grace Lee,ENG401,78.1
14,Grace Lee,PHY301,93.1
16,Diana Wilson,PHY301,61.2
2,Jack Taylor,BIO501,82.1
2,Jack Taylor,PHY301,75.1
2,Jack Taylor,PHY301,90.1
24,Alice Smith,CS101,67.8
24,Alice Smith,MATH201,63.0
24,Alice Smith,PHY301,92.7
25,Grace Lee,BIO501,66.1
25,Grace Lee,BIO501,67.1
25,Grace Lee,CS101,70.7
30,Frank Miller,BIO501,61.8
34,Alice Smith,BIO501,68.6
38,Emma Davis,MATH201,90.3
38,Emma Davis,PHY301,82.2
38,Emma Davis,PHY301,85.5
4,Charlie Brown,ENG401,91.1
4,Charlie Brown,MATH201,94.4
41,Henry Clark,CS101,98.6
41,Henry Clark,ENG401,87.8
43,Bob Johnson,MATH201,63.7
44,Jack Taylor,ENG401,94.2
44,Jack Taylor,MATH201,65.2
```

Image showing final result

The same thing can be done using Dataproc cluster:

Following screen shots will show the detail steps:



```
Creating directory '/home/bidhan_pokhrel_dubai_gmail_com'.
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ la
.bash_logout .bashrc .cache .profile generate_tables.py mapper.py reducer.py
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ ls
generate_tables.py mapper.py reducer.py
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ chmod +x mapper.py
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ chmod +x reducer.py
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ cat table_a.csv table_b.csv |python mapper.py>mapout.csv
cat: table_a.csv: No such file or directory
cat: table_b.csv: No such file or directory
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ python generate_tables.py
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ ls
generate_tables.py mapout.csv mapper.py reducer.py table_a.csv table_b.csv
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ cat table_a.csv table_b.csv |python mapper.py>mapout.csv
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ cat mapout.csv| sort -k1,1n |python reducer.py>redout.csv
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$
```

```
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ cat mapout.csv
1      A      Charlie Brown      1999-08-23
3      A      Jack Taylor      1995-01-17
5      A      Diana Wilson      2000-01-28
6      A      Isabella Adams      1997-07-06
9      A      Diana Wilson      1998-11-23
10     A      Charlie Brown      1995-06-04
11     A      Charlie Brown      2000-03-17
```

```
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ cat redout.csv
StudentId,Name,CourseId,Grade
1,Charlie Brown,BIO501,64.4
1,Charlie Brown,BIO501,94.9
1,Charlie Brown,MATH201,67.1
3,Jack Taylor,CS101,90.4
```

Now time to run the same thing in Hadoop system:

#Making directory named as input inside Hadoop:

```
hadoop fs -mkdir -p /user/bidhan_pokhrel_dubai_gmail_com/challenge4/input
```

#copy the files named table_a.csv and table_b.csv in Hadoop input folder:

```
hadoop fs -put table_a.csv table_b.csv /user/bidhan_pokhrel_dubai_gmail_com/challenge4/input
```

#running mapreduce:

```
hadoop jar /usr/lib/hadoop/hadoop-streaming.jar \
```

```
-files mapper.py, reducer.py \
```

```
-mapper mapper.py \
```

```
-reducer reducer.py \
```

```
-input /user/bidhan_pokhrel_dubai_gmail_com/challenge4/input \
```

```
-output /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output \
```

```
-jobconf mapreduce.job.reduces=1
```

```
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ hadoop jar /usr/lib/hadoop/hadoop-streaming.jar \
> -files mapper.py, reducer.py \
> -mapper mapper.py \
> -reducer reducer.py \
> -input /user/bidhan_pokhrel_dubai_gmail_com/challenge4/input \
> -output /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output \
> -jobconf mapreduce.job.reduces=1
```

```
2025-07-09 09:40:36,570 INFO mapreduce.Job: Job job_1752053480600_0001 running in uber mode : false
2025-07-09 09:40:36,571 INFO mapreduce.Job: map 0% reduce 0%
2025-07-09 09:40:44,668 INFO mapreduce.Job: map 10% reduce 0%
2025-07-09 09:40:49,705 INFO mapreduce.Job: map 40% reduce 0%
2025-07-09 09:40:55,741 INFO mapreduce.Job: map 50% reduce 0%
2025-07-09 09:40:59,766 INFO mapreduce.Job: map 70% reduce 0%
2025-07-09 09:41:00,771 INFO mapreduce.Job: map 80% reduce 0%
2025-07-09 09:41:08,813 INFO mapreduce.Job: map 100% reduce 0%
2025-07-09 09:41:15,856 INFO mapreduce.Job: map 100% reduce 100%
2025-07-09 09:41:17,875 INFO mapreduce.Job: Job job_1752053480600_0001 completed successfully
2025-07-09 09:41:17,963 INFO mapreduce.Job: Counters: 55
File System Counters
```

```
Peak Reduce Virtual Memory (bytes)=4731523072
Shuffle Errors
BAD_ID=0
CONNECTION=0
IO_ERROR=0
WRONG_LENGTH=0
WRONG_MAP=0
WRONG_REDUCE=0
File Input Format Counters
Bytes Read=16730
File Output Format Counters
Bytes Written=2371
2025-07-09 09:41:17,963 INFO streaming.StreamJob: Output directory: /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$
```

```
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ hadoop fs -ls /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output/
Found 2 items
-rw-r--r-- 2 bidhan_pokhrel_dubai_gmail_com hadoop 0 2025-07-09 09:41 /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output/_SUCCESS
-rw-r--r-- 2 bidhan_pokhrel_dubai_gmail_com hadoop 2371 2025-07-09 09:41 /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output/part-00000
bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ hadoop fs -cat /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output/part*
StudentId,Name,CourseId,Grade
1,Charlie Brown,MATH201,67.1
1,Charlie Brown,BIO501,64.4
1,Charlie Brown,BIO501,94.9
10,Charlie Brown,CS101,70.3
```



```

bidhan_pokhrel_dubai_gmail_com@challenge4-m:~$ hadoop fs -cat /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output/part*
StudentId,Name,CourseId,Grade
1,Charlie Brown,MATH201,67.1
1,Charlie Brown,BIO501,64.4
1,Charlie Brown,BIO501,94.9
10,Charlie Brown,CS101,70.3
11,Charlie Brown,ENG401,88.3
14,Isabella Adams,PHY301,70.7
17,Charlie Brown,ENG401,99.3
17,Charlie Brown,BIO501,89.4
17,Charlie Brown,ENG401,83.7
18,Alice Smith,BIO501,76.6
18,Alice Smith,CS101,97.6
20,Bob Johnson,MATH201,76.7
20,Bob Johnson,ENG401,76.3
21,Jack Taylor,ENG401,96.1
21,Jack Taylor,ENG401,86.5
24,Charlie Brown,ENG401,73.0
24,Charlie Brown,BIO501,95.7
26,Emma Davis,CS101,79.0
26,Emma Davis,PHY301,85.9
29,Emma Davis,ENG401,76.4
29,Emma Davis,CS101,83.4
3,Jack Taylor,CS101,90.4
32,Charlie Brown,BIO501,69.9
35,Henry Clark,CS101,96.4
36,Isabella Adams,MATH201,77.6
36,Isabella Adams,CS101,70.4
37,Diana Wilson,BIO501,90.2
38,Emma Davis,CS101,74.4
42,Isabella Adams,PHY301,68.9
42,Isabella Adams,ENG401,92.9
48,Bob Johnson,BIO501,89.1
48,Bob Johnson,BIO501,95.5
5,Diana Wilson,BIO501,76.8
5,Diana Wilson,PHY301,81.3
52,Frank Miller,BIO501,93.4
52,Frank Miller,ENG401,86.9
52,Frank Miller,PHY301,71.3
55,Charlie Brown,MATH201,64.4
58,Bob Johnson,BIO501,95.4
6,Isabella Adams,BIO501,82.5

```

See output: `hadoop fs -cat /user/bidhan_pokhrel_dubai_gmail_com/challenge4/output/part*`

Task 2:

For this task we need to generate the dataset first. To do so Gen AI was very useful.

The prompt that I gave to Gen AI : give me the code in python to generate the dataset of more than 2GB in csv format. the data set must have different rows and columns. if possible give me the codes to create a data set for health related data.

give me the code in python to generate the dataset of more than 2GB in csv format.
the data set must have different rows and columns.
if possible give me the codes to create a data set for health related data.





bidhan-pokhrel Rename Generate 2GB +data.py to Generate_largedata.py

Code

Blame

76 lines (65 loc) · 2.69 KB

```
1  import pandas as pd
2  import numpy as np
3  from faker import Faker
4  import random
5  from datetime import datetime, timedelta
6  import os
7
8  # Initialize Faker for realistic data
9  fake = Faker()
10
11 # Define parameters
12 ✓ columns = [
13     'PatientID', 'FirstName', 'LastName', 'DOB', 'Gender',
14     'BloodPressureSystolic', 'BloodPressureDiastolic', 'HeartRate',
15     'CholesterolLevel', 'BloodSugarLevel', 'Diagnosis',
16     'AdmissionDate', 'Weight', 'Height', 'BMI'
17 ]
18
19 # Possible values for categorical fields
20 genders = ['Male', 'Female', 'Other']
21 diagnoses = ['Hypertension', 'Diabetes', 'Heart Disease', 'Asthma', 'None', 'COPD', 'Obesity']
22
23 # Function to generate a single row of health data
24 ✓ def generate_health_row():
25     weight = random.uniform(50, 120) # Weight in kg
26     height = random.uniform(150, 200) # Height in cm
```

```
26     height = random.uniform(150, 200) # Height in cm
27     bmi = round(weight / ((height / 100) ** 2), 2) # Calculate BMI
28     dob = fake.date_of_birth(minimum_age=18, maximum_age=90)
29     admission_date = fake.date_between(start_date='-2y', end_date='today')
30
31     return {
32         'PatientID': fake.uuid4(),
33         'FirstName': fake.first_name(),
34         'LastName': fake.last_name(),
35         'DOB': dob.strftime('%Y-%m-%d'),
36         'Gender': random.choice(genders),
37         'BloodPressureSystolic': random.randint(90, 180),
38         'BloodPressureDiastolic': random.randint(60, 120),
39         'HeartRate': random.randint(60, 100),
40         'CholesterolLevel': random.randint(120, 300),
41         'BloodSugarLevel': random.randint(70, 200),
42         'Diagnosis': random.choice(diagnoses),
43         'AdmissionDate': admission_date.strftime('%Y-%m-%d'),
44         'Weight': round(weight, 2),
45         'Height': round(height, 2),
46         'BMI': bmi
47     }
48
49     # Target file size: >2GB (approx 2,000,000,000 bytes)
50     # Estimate row size: ~200-300 bytes per row (based on sample data)
51     # Target ~10 million rows for ~2-3 GB
52     num_rows_per_chunk = 100000 # Process in chunks to manage memory
53     num_chunks = 250 # Adjust to achieve >2GB
54     output_file = 'health_dataset.csv'
```

```

55
56     # Remove existing file if it exists
57     if os.path.exists(output_file):
58         os.remove(output_file)
59
60     # Generate and write data in chunks
61     for chunk in range(num_chunks):
62         # Generate chunk of data
63         data = [generate_health_row() for _ in range(num_rows_per_chunk)]
64         df = pd.DataFrame(data, columns=columns)
65
66         # Write to CSV (append mode after first chunk)
67         mode = 'w' if chunk == 0 else 'a'
68         header = True if chunk == 0 else False
69         df.to_csv(output_file, mode=mode, header=header, index=False)
70
71         print(f"Written chunk {chunk + 1}/{num_chunks}")
72
73     print(f"Dataset generated at: {output_file}")
74     # Check file size
75     file_size = os.path.getsize(output_file) / (1024 ** 3) # Size in GB
76     print(f"Generated file size: {file_size:.2f} GB")

```

Here in this code we can modify the line number 53, num_chunks, if we increase the number the dataset will be larger and if the chunk size is smaller then the dataset size will also be smaller. So, we can customize as per our needs.

```

C:\Users\Dell\Downloads>pip install faker
Collecting faker
  Using cached faker-37.4.0-py3-none-any.whl.metadata (15 kB)
Requirement already satisfied: tzdata in c:\users\dell\appdata\local\programs\python\python313\lib\site-packages (from faker) (2025.1)
Using cached faker-37.4.0-py3-none-any.whl (1.9 MB)
Installing collected packages: faker
Successfully installed faker-37.4.0

[notice] A new release of pip is available: 25.0.1 -> 25.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
C:\Users\Dell\Downloads>

```

```

C:\Users\Dell\Downloads>python Generate_largedata.py
Written chunk 1/250
Written chunk 2/250
Written chunk 3/250

```



Then, I uploaded the dataset to my google bucket.

Bidhan Pokhrel buckets X Search 19 ?

Object details Learn

Buckets > bdnlargedata > health_dataset.csv

Live object Version history

Download Edit metadata Edit access Delete

Overview

Type	text/csv
Size	2.5 GB
Created	8 Jul 2025, 08:09:31
Last modified	8 Jul 2025, 08:09:31
Storage class	Standard
Custom time	—
Public URL ?	https://storage.googleapis.com/bdnlargedata/health_dataset.csv
Authenticated URL ?	https://storage.cloud.google.com/bdnlargedata/health_dataset.csv
gsutil URI ?	gs://bdnlargedata/health_dataset.csv
Permissions	
Public access	Not public
Protection	
Version history ?	—
Retention expiry time ?	None

Now time to generate the Spark code.

Gen AI prompt that I used to generate the Pyspark code:

Generate_largedata.py
PY 2.69KB

generate a Spark code in Python for the dataset that will be generated using the provided code.
the spark code will be used to select the random 100 rows from the dataset.
also the location of the dataset is : gs://bdnlargedata/health_dataset.csv

the code should run on data proc pyspark on my cloud machine.



To run Spark on my cluster, I need to go to Web Interferences> Jupyter.

Bidhan Pokhrel

data proc

×

Q

/ Cluster: challenge4 / Interfaces

Cluster details

SUBMIT JOB

REFRESH

START

STOP

DELETE

VIEW LOG

MONITORING

JOBS

VM INSTANCES

CONFIGURATION

WEB INTERFACES

SSH tunnel

Create an SSH tunnel to connect to a web interface

Component gateway

Provides access to the web interfaces of default and selected optional components on the cluster. [Learn more](#)

[YARN ResourceManager](#)

[MapReduce Job History](#)

[Spark History Server](#)

[HDFS NameNode](#)

[YARN Application Timeline](#)

[Tez](#)

[Jupyter](#)

[JupyterLab](#)

bidhan-pokhrel > challenge4

Sign out

jupyter

Files

Running

IPython Clusters

Nbextensions

Select items to perform actions on them.

Upload

New

0

/

Name

Last Modified

File size

GCS

19 hours ago

Local Disk

3 minutes ago

bidhan-pokhrel > challenge4

Sign out

jupyter

Files

Running

IPython Clusters

Nbextensions

Select items to perform actions on them.

Upload

New

0

/ GCS

Name

Last Modified

File size

..

seconds ago

Untitled.ipynb

19 hours ago

bidhan-pokhrel > challenge4

Jupyter Untitled (autosaved)

File Edit View Insert Cell Kernel Widgets Help Trusted PySpark O

Run Code nbdiff

```
In [1]:
from pyspark.sql import SparkSession
import pyspark.sql.functions as F

# Step 1: Initialize Spark Session with GCS connector
spark = SparkSession.builder \
    .appName("RandomRowSelectionFromGCS") \
    .config("spark.hadoop.fs.gs.impl", "com.google.cloud.hadoop.fs.gcs.GoogleHadoopFileSystem") \
    .config("spark.hadoop.google.cloud.auth.service.account.enable", "true") \
    .getOrCreate()

# Step 2: Load the dataset from GCS
dataset_path = "gs://bdnlargedata/health_dataset.csv"
df = spark.read.option("header", "true").option("inferSchema", "true").csv(dataset_path)

# Step 3: Randomly select exactly 100 rows
random_rows = df.orderBy(F.rand(seed=42)).limit(100)

# Step 4: Display the results
random_rows.show(100, truncate=False)

# Optional: Save results to GCS
output_path = "gs://bdnlargedata/output/random_rows"
random_rows.write.csv(output_path, mode="overwrite", header=True)

# Step 5: Stop the Spark session
spark.stop()
```

25/07/08 15:57:02 WARN SparkSession: Using an existing Spark session; only runtime SQL configurations will take effect.

PatientID	FirstName	LastName	DOB	Gender	BloodPressureSystolic	BloodPressureDiastolic	HeartRate	CholesterolLevel	BloodSugarLevel	Diagnosis	AdmissionDate	Weight	Height	BMI
c14c0499-d44a-4d98-acd9-487378df9ab9	Caroline	Rowe	2005-08-30 00:00:00	Other	96									77
474fd362-8a0a-4c09-a905-826ddb451b5	Michelle	Hardin	1944-07-21 00:00:00	Male	170									83
29d76c22-7aec-4b22-aef7-746ce8d406b3	Jeffrey	Watson	1946-08-04 00:00:00	Female	151									85
b4e25568-7543-4ce6-97bd-9435ededb870	Steven	Harper	1940-08-25 00:00:00	Female	150									78
ba602e41-2ffa-4668-a1de-87297f449432	Brian	Spencer	1949-01-24 00:00:00	Male	110									109
1e3b011d-9cab-4ed9-a3e5-e9e8a27bb3837	Linda	Mcroy	1981-10-13 00:00:00	Female	143									100

```
spark.stop()

178 | 207 | 155 | Asthma | 2023-05-21 00:00:00 | 99.75 | 196.00 | 25.79 | 101
13914e64f-da60-4120-a0c3-75c74e72466c | Ian | Stanley | 1958-12-07 00:00:00 | Female | 158 | 101
85 | 262 | 147 | None | 2023-09-30 00:00:00 | 59.89 | 167.64 | 21.31 | 93
d296988d-4faa-4eea-a791-88d612fca06c | Thomas | Baker | 1970-03-16 00:00:00 | Male | 99 | 93
73 | 250 | 149 | Asthma | 2024-12-03 00:00:00 | 93.15 | 169.97 | 32.24 | 94
2b26a41c-cf9e-4806-94a9-225d0082a4b2 | Kathy | Lewis | 1947-11-17 00:00:00 | Female | 174 | 94
89 | 120 | 85 | None | 2024-08-30 00:00:00 | 89.03 | 158.23 | 35.56 | 75
c849b9ea-3e80-441c-be44-14e20b155c69 | Kathryn | Gilbert | 1963-04-21 00:00:00 | Male | 101 | 75
76 | 216 | 148 | Obesity | 2024-03-19 00:00:00 | 73.75 | 192.57 | 19.89 | 113
f6a6f0c3-b8f0-42dd-8b29-dddab4b049ba | Jessica | Massey | 1981-06-04 00:00:00 | Male | 121 | 113
90 | 267 | 83 | Hypertension | 2025-06-14 00:00:00 | 72.47 | 168.84 | 25.42 | 75
146e476a-0c3e-4901-a380-e059a5e4a357 | James | Fleming | 1975-12-08 00:00:00 | Female | 149 | 75
99 | 291 | 154 | Heart Disease | 2023-12-06 00:00:00 | 96.41 | 197.88 | 24.62 | 66
f0442a87-aa92-434f-bf88-77294113aa3b | Denise | Davis | 2005-07-08 00:00:00 | Other | 101 | 66
87 | 167 | 104 | Asthma | 2025-07-03 00:00:00 | 78.04 | 185.35 | 22.72 |
```

In []:

Task3:

For task 3: I used the temp.csv for dataset and map2.py and reducer.py from our teams.

And I uploaded those three files to Google Bucket where the result will also be stored inside the output folder as specified in our arguments.

The screenshot shows the Google Cloud Storage interface for a bucket named 'bdntemp'. The bucket is located in 'us (multiple regions in United States)', has a 'Standard' storage class, 'Not public' access, and 'Soft delete' protection. The 'Objects' tab is selected, displaying a list of files and folders. The table below summarizes the contents:

Name	Size	Type	Created	Storage class	Last m
map2.py	271 B	text/x-python	9 Jul 2025, 16:32:23	Standard	9 Jul 2
output/	—	Folder	—	—	—
reducer2.py	746 B	text/x-python	8 Jul 2025, 19:21:12	Standard	8 Jul 2
temp.csv	221 B	text/csv	8 Jul 2025, 19:21:12	Standard	8 Jul 2

Job Creation:

The screenshot shows the 'Submit a job' form in the Google Cloud Dataproc console. The form is for a job named 'challenge4' in the 'us-central1' region, using the 'challenge4' cluster. The job type is 'Hadoop'. The main class or JAR is 'file:///usr/lib/hadoop/hadoop-streaming.jar'. The form also includes fields for 'Jar files' and 'Archive files', both of which are currently empty.

Job ID *
challenge4

Region *
us-central1
Specifies the Cloud Dataproc regional service, which determines what clusters are available.

Cluster *
challenge4

Job type *
Hadoop

Main class or JAR *
file:///usr/lib/hadoop/hadoop-streaming.jar
The fully qualified name of a class in a provided or standard JAR file, for example, com.example.wordcount, or a provided JAR file to use the main class of that JAR file

Jar files
JAR files are included in the CLASSPATH. Can be a GCS file with the gs:// prefix, an HDFS file on the cluster with the hdfs:// prefix, or a local file on the cluster with the file:// prefix.

Archive files
Archive files are extracted in the Spark working directory. Can be a GCS file with the gs://

Clusters

Clusters

Jobs

Workflows

Autoscaling policies

Serverless

Batches

Interactive Sessions

Session Templates

Metastore Services

Metastore

Release Notes

Main class or jar *

file:///usr/lib/hadoop/hadoop-streaming.jar

The fully qualified name of a class in a provided or standard jar file, for example, com.example.wordcount, or a provided jar file to use the main class of that jar file

Jar files

Jar files are included in the CLASSPATH. Can be a GCS file with the gs:// prefix, an HDFS file on the cluster with the hdfs:// prefix, or a local file on the cluster with the file:// prefix.

Archive files

Archive files are extracted in the Spark working directory. Can be a GCS file with the gs:// prefix, an HDFS file on the cluster with the hdfs:// prefix, or a local file on the cluster with the file:// prefix. Supported file types: jar, tar, tar.gz, tgz, zip.

Arguments

files

gs://bdntemp/map2.py

gs://bdntemp/reducer2.py

-mapper

python3 map2.py

-reducer

python3 reducer2.py

-input

gs://bdntemp/temp.csv

-output

gs://bdntemp/output

Press <Return> to add more arguments

Additional arguments to pass to the main class. Press Return after each argument.

After this we can click submit and the task will execute the mapper and reducer for the given file on Hadoop system.

Google Cloud

Bidhan Pokhrel

data

Search

Dataproc / Jobs / Job: challenge4 / Monitoring

Overview

Notebooks/IDE

BigQuery Studio

Workbench

Clusters

Jobs

Workflows

Autoscaling policies

Serverless

Batches

Interactive Sessions

Session Templates

Metastore Services

Metastore

Release Notes

Job details

CLONE

DELETE

STOP

REFRESH

Job ID

challenge4

Job UUID

f4384c73-45f6-4a25-ba6e-aaf5bca1093a

Type

Dataproc Job

Status

Running

MONITORING

CONFIGURATION

The charts below represent the metrics from the cluster this job ran on, scoped to the time that this job was running. It is possible for more than one job to run on a cluster at a time, so these metrics may not reflect this job's resource usage accurately. Metrics for a job may lag behind the job run by several minutes.

SAVE AS DASHBOARD

RESET ZOOM

1 hour

6 hours

12 hours

1 day

2 days

4 days

7 days

14 days

30 days

Custom 9:57 PM - 10:02 PM

Output

LINE WRAP: OFF

Press Alt+F1 for Accessibility Options.

WRONG_LENGTH=0

WRONG_MAP=0

WRONG_REDUCE=0

File Input Format Counters

Bytes Read=1130

File Output Format Counters

Bytes Written=46

2025-07-08 16:18:41,417 INFO streaming.StreamJob: Output directory: gs://bdntemp/output

Pending output is streaming

Job challenge4 successfully submitted

EQUIVALENT COMMAND LINE

Google Cloud

Bidhan Pokhrel

data

Search

Dataproc / Jobs / Job: challenge4 / Monitoring

Overview

Notebooks/IDE

BigQuery Studio

Workbench

Clusters

Jobs

Workflows

Autoscaling policies

Serverless

Batches

Interactive Sessions

Session Templates

Metastore Services

Metastore

Release Notes

Job details

CLONE

DELETE

STOP

REFRESH

Job ID

challenge4

Job UUID

f4384c73-45f6-4a25-ba6e-aaf5bca1093a

Type

Dataproc Job

Status

Succeeded

MONITORING

CONFIGURATION

The charts below represent the metrics from the cluster this job ran on, scoped to the time that this job was running. It is possible for more than one job to run on a cluster at a time, so these metrics may not reflect this job's resource usage accurately. Metrics for a job may lag behind the job run by several minutes.

Output

LINE WRAP: OFF

WRONG_LENGTH=0

WRONG_MAP=0

WRONG_REDUCE=0

File Input Format Counters

Bytes Read=1130

File Output Format Counters

Bytes Written=46

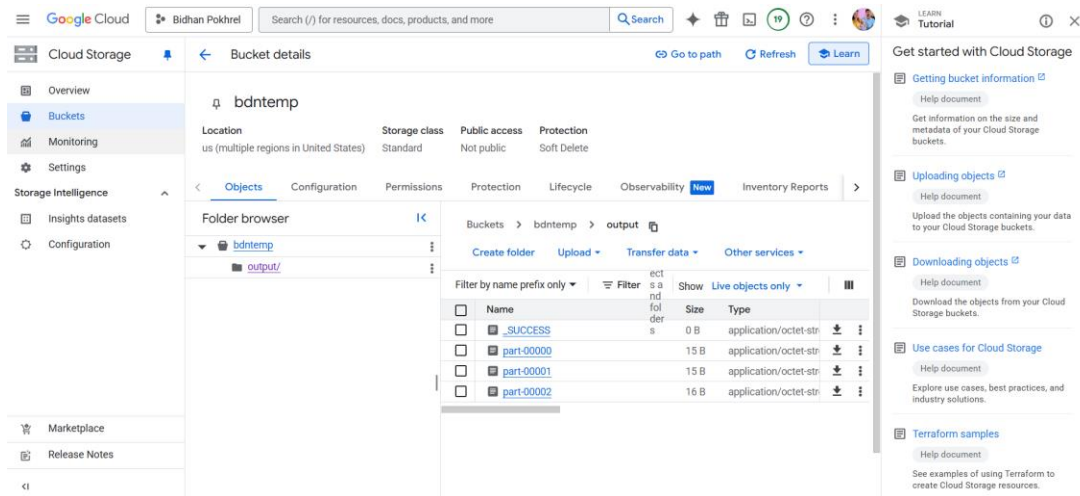
2025-07-08 16:18:41,417 INFO streaming.StreamJob: Output directory: gs://bdntemp/output

Output is complete

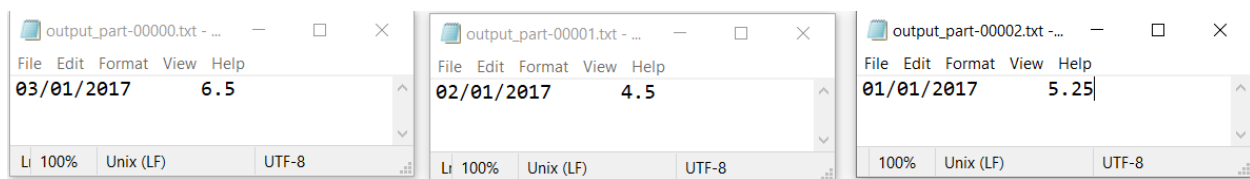
Job challenge4 successfully submitted

EQUIVALENT COMMAND LINE

After successfully running the job output will be saved in the predefined output folder. Here, we have not defined the number of reducer so system automatically creates the reducers by default setting. Here, we can see the output are in three parts, names as: part-00000, part-00001, part-00002.



We can download and see the output as below:



This shows the daily average temperature from the input data.

If we had limited the reducer to one, then the result would be something like below:

01/01/2017 5.25

02/01/2017 4.5

03/01/2017 6.5